

REPORT

자유주제 프로젝트



과 목 명 : 영상정보처리

담당교수 : 김하연

학 과 : 소프트웨어학과

학 번 : 32227297

이 름 : 마상위

제 출 일 : 2026.06.14

프로젝트 개요

나는 당구를 칠 때 내가 친 수구가 어떤 동선으로 굴렀는지를 위에서 내려다본 그림으로 보고 싶었다. 옆에서 비스듬히 찍힌 중계 영상으로는 쿠션에 맞고 꺾이는 각이 원근 때문에 실제보다 찌그러져 보여서, "이 공이 진짜로는 어떤 경로를 그렸나"를 직관적으로 알기 어렵다. 그래서 비스듬히 찍힌 한 영상에서 수구만 골라내 추적하고, 그 궤적을 진짜 직사각형 테이블 위에 퍼서 보여 주는 도구를 만들기로 했다.

입력 → 파이프라인 → 출력

당구 영상 1프레임

└─[층1] 흰 공 검출 : HSV 이진화 → ROI → 모폴로지(열기·닫기) → 컨투어 면적·중형비 필터

└─ 수구 중심좌표 (x, y)

└─[시간축] 직전 위치에 최근접한 공으로 연결 → 궤적 누적

└─[층2] 호모그래피로 (x,y)를 탭뷰 좌표로 사상

└─ 출력 : 원본 영상 위 궤적 + 탭뷰 위 궤적 (동시에)

사용한 층

층1 (픽셀 값)— HSV 이진화 + 모폴로지 + 컨투어 면적·중형비 분석으로 흰 수구를 분리.

층2 (픽셀 위치)— 빈 테이블 코너 4점으로 호모그래피를 구해, 비스듬히 찍힌 테이블을 정면 직사각형 탭뷰로 펴

구현과정

```
# ===== 설정 & 헬퍼 =====
import cv2, numpy as np, os
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm

VIDEO_PATH = "video1.mp4"
START = 112 # 추적 시작 프레임
CLEAN_FRAME = 360 # 빈 테이بل 프레임(코너용)
TABLE_W, TABLE_H = 1000, 500
MAX_FRAMES = 400 # 안전장치: 무한루프 방지
WHITE_LO = np.array([0, 0, 190]); WHITE_HI = np.array([180, 45, 255])
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))

# 한글 폰트 설정
def set_korean_font():
    candidates = ["Malgun Gothic", "AppleGothic", "NanumGothic",
                  "NanumBarunGothic", "Noto Sans CJK KR", "Noto Sans KR"]
    installed = {f.name for f in fm.fontManager.ttflist}
    for name in candidates:
        if name in installed:
            plt.rcParams["font.family"] = name
            print("한글 폰트:", name)
            break
    else:
        print("한글 폰트 못 찾음 - 제목만 영어로 바뀌도 됨")
    plt.rcParams["axes.unicode_minus"] = False

set_korean_font()

def show(img, title="", gray=False, figsize=(7, 4)):
    """이미지 한 장 띄우기"""
    plt.figure(figsize=figsize)
    plt.imshow(img, cmap="gray") if gray else plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
```

```
def show(img, title="", gray=False, figsize=(7, 4)):
    """이미지 한 장 띄우기"""
    plt.figure(figsize=figsize)
    plt.imshow(img, cmap="gray") if gray else plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
    plt.title(title); plt.axis("off"); plt.show()

def show_row(imgs, titles, gray=False, figsize=(16, 4)):
    """여러 장 한 줄로 비교 (before/after용)"""
    fig, axes = plt.subplots(1, len(imgs), figsize=figsize)
    for ax, im, t in zip(axes, imgs, titles):
        ax.imshow(im, cmap="gray") if gray else ax.imshow(cv2.cvtColor(im, cv2.COLOR_BGR2RGB))
        ax.set_title(t); ax.axis("off")
    plt.tight_layout(); plt.show()

def save_video(frames, filename, fps=30):
    h, w = frames[0].shape[:2]
    out = cv2.VideoWriter(filename, cv2.VideoWriter_fourcc(*'mp4v'), fps, (w, h))
    for f in frames: out.write(f)
    out.release()
    print("[저장]", os.path.abspath(filename))

def get_frame(idx):
    cap = cv2.VideoCapture(VIDEO_PATH); cap.set(cv2.CAP_PROP_POS_FRAMES, idx)
    ok, f = cap.read(); cap.release()
    return f
```

한글 폰트: Malgun Gothic

영상 처리를 직접 하는 코드가 아니라, 뒤 단계가 공통으로 쓸 값과 도구를 미리 모아둔 곳이다.

설정 상수 (이 영상에 맞춰 내가 정한 값)

— START=112(추적 시작 프레임)

CLEAN_FRAME=360(코너용 빈 테이블 프레임)

TABLE_W·H=1000·500(탑뷰 크기, 2:1)

WHITE_LO/HI(흰 공 HSV 범위)

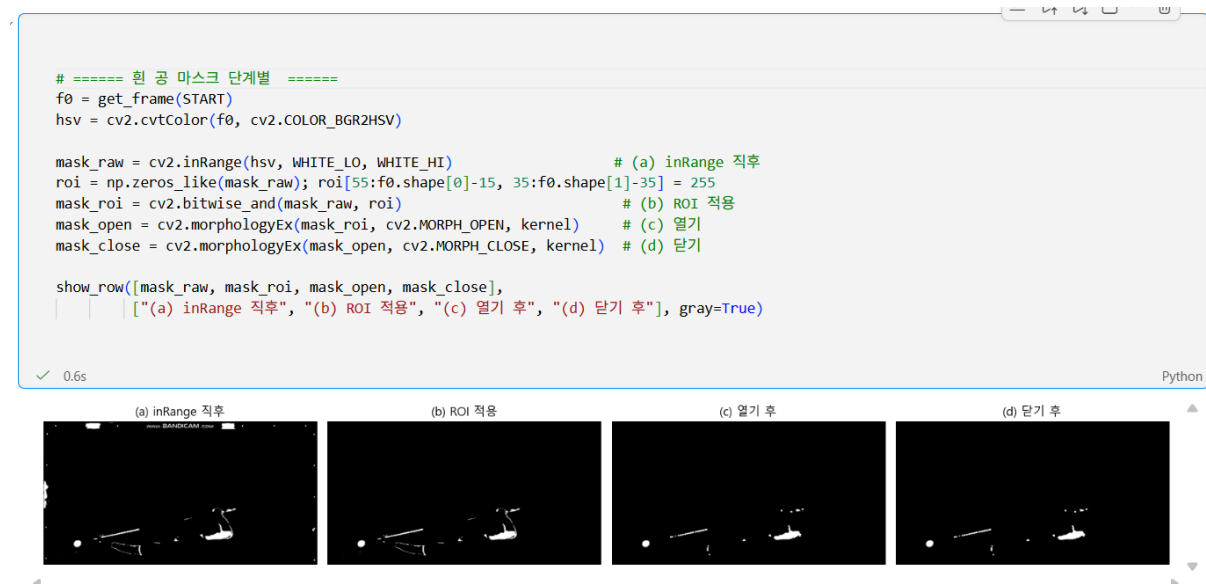
kernel(5×5 타원).

다른 영상엔 다시 정해야 하는 튜닝 값들을 한곳에 모아 둔 것.

MAX_FRAMES=400 — 추적 루프가 끝까지 안 멈추는 사고를 막는 안전장치(무한루프 방지).

set_korean_font() — 그림 제목 한글이 깨지지 않게 OS별 폰트 후보 중 설치된 첫 것을 잡아주는 함수. 못 찾으면 에러 없이 넘어간다. 알고리즘이 아니라 표시용 플러밍이다.

show() — 이미지를 띄우는 헬퍼. 핵심은 cvtColor(BGR2RGB) 한 줄 — OpenCV는 색을 BGR, matplotlib는 RGB로 읽어서 안 바꾸면 색이 뒤집힌다.



흰 공 마스크가 만들어지는 4단계

(a) inRange 직후 → (b) ROI 적용 → (c) 열기(open) → (d) 닫기(close)

(a) 흰색 HSV 범위로 이진화하면 공만이 아니라 위쪽 www.BANDICAM.com 워터마크 글자와 길쭉한 큐대까지 흰색으로 같이 잡힌다.

(b) roi로 화면 위쪽 55px와 바깥 테두리를 잘라내자 워터마크 글자가 통째로 사라졌다. 워터마크가 항상 같은 위치(상단)라서 위치 기반으로 자르는 게 가장 확실했다.

(c) 열기로 침식→팽창을 하니 큐대처럼 가는 막대와 점 노이즈가 끊겨 없어졌다. 공은 두툼해서 살아남는다.

(d) 닫기로 팽창→침식을 하니 공 표면 하이라이트 때문에 생긴 내부 구멍이 메워져 한 덩어리가 됐다. 이 상태라야 컨투어 면적이 안정적으로 나온다.

```
# ===== 컨투어 면적·비율 필터링 전후 =====
cnts, _ = cv2.findContours(mask_close, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

vis_all = f0.copy() # (a) 필터링 전
for c in cnts:
    x, y, bw, bh = cv2.boundingRect(c)
    cv2.rectangle(vis_all, (x, y), (x+bw, y+bh), (0, 255, 255), 1)

vis_kept = f0.copy() # (b) 필터링 후
for c in cnts:
    a = cv2.contourArea(c)
    if not (150 < a < 2600): continue
    x, y, bw, bh = cv2.boundingRect(c)
    if not (0.55 < bw/bh < 1.8): continue
    cv2.rectangle(vis_kept, (x, y), (x+bw, y+bh), (0, 230, 0), 2)
    cv2.putText(vis_kept, f"A={int(a)} r={bw/bh:.2f}", (x, y-5),
                cv2.FONT_HERSHEY_SIMPLEX, 0.4, (0, 230, 0), 1)

show_row([vis_all, vis_kept], ["(a) 모든 컨투어", "(b) 면적·비율 통과만"])

✓ 0.2s Python
```



```
# ===== 흰 공 검출 함수 정의 =====
def find_white_balls(frame):
    """중1: HSV 이진화 + ROI + 모폴로지 열기/닫기 + 컨투어 + 면적·비율 필터"""
    h, w = frame.shape[:2]
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    mask = cv2.inRange(hsv, WHITE_LO, WHITE_HI)
    roi = np.zeros_like(mask); roi[55:h-15, 35:w-35] = 255
    mask = cv2.bitwise_and(mask, roi)
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
    cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    balls = []
    for c in cnts:
        a = cv2.contourArea(c)
        if not (150 < a < 2600): continue
        x, y, bw, bh = cv2.boundingRect(c)
        if not (0.55 < bw/bh < 1.8): continue
        balls.append((a, (x + bw // 2, y + bh // 2)))
    balls.sort(key=lambda b: -b[0])
    return [c for _, c in balls]

✓ 0.0s Python
```

컨투어 필터링 — 전 vs 통과만

(a) 모든 컨투어 / (b) 면적·중형비 통과만

(a) 마스크에서 검출된 컨투어를 전부 박스 치면 수구뿐 아니라 큐대 영역과 손/장갑 근처까지 박스가 잡힌다.

(b) $150 < \text{면적} < 2600$ 과 $0.55 < \text{가로/세로} < 1.8$ 두 조건을 통과한 건 흰 수구 단 하나다. 큐대는 길쭉해서 종횡비 조건에서 탈락한다

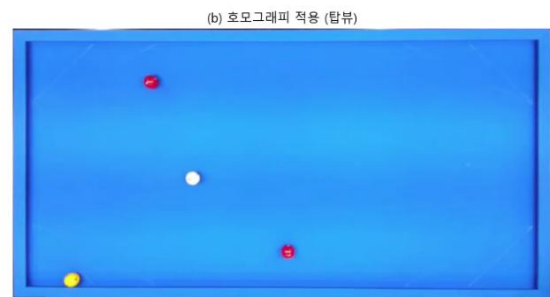
```
# ===== 호모그래피 전후 =====
def order_corners(pts):
    rect = np.zeros((4, 2), np.float32)
    s = pts.sum(1); d = np.diff(pts, 1).flatten()
    rect[0] = pts[np.argmin(s)]; rect[2] = pts[np.argmax(s)] # TL, BR
    rect[1] = pts[np.argmin(d)]; rect[3] = pts[np.argmax(d)] # TR, BL
    return rect

clean = get_frame(CLEAN_FRAME)
blue = cv2.inRange(cv2.cvtColor(clean, cv2.COLOR_BGR2HSV),
    np.array([95, 90, 60]), np.array([125, 255, 255]))
blue = cv2.morphologyEx(blue, cv2.MORPH_CLOSE, np.ones((15, 15), np.uint8))
cnts, _ = cv2.findContours(blue, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
src = order_corners(max(cnts, key=cv2.contourArea).reshape(-1, 2).astype(np.float32))
dst = np.float32([[0, 0], [TABLE_W, 0], [TABLE_W, TABLE_H], [0, TABLE_H]])
Hmat = cv2.getPerspectiveTransform(src, dst)
table_bg = cv2.warpPerspective(clean, Hmat, (TABLE_W, TABLE_H))

vis_corners = clean.copy()
for i, p in enumerate(src.astype(int)):
    cv2.circle(vis_corners, tuple(p), 8, (0, 0, 255), -1)
    cv2.putText(vis_corners, ["TL", "TR", "BR", "BL"][i],
        tuple(p + np.array([10, 0])),
        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
cv2.polylines(vis_corners, [src.astype(int)], True, (0, 255, 0), 2)

show_row([vis_corners, table_bg], ["(a) 코너 검출 (원본)", "(b) 호모그래피 적용 (탭뷰)"])

to_top = lambda p: tuple(cv2.perspectiveTransform(
    np.array([p]), np.float32, Hmat).reshape(2).astype(int))
```



코너 검출 → 호모그래피 탭뷰

(a) 빈 테이블 코너 검출(원본) / (b) 호모그래피 적용(탭뷰)

(a) 공·사람이 없는 빈 프레임(360번)에서 파란 영역의 가장 큰 컨투어를 잡고, 그 컨투어 점들 중 합/차가 극값인 네 점을 코너(빨간 점)로 정렬했다. 카메라가 고정이라 코너는 한 번만 찾으면 영상 내내 유효하다.

(b) 그 네 점을 1000×500 직사각형으로 보내는 호모그래피로 테이블을 펴니, 비스듬한 사다리꼴이 정면 직사각형 탭뷰가 됐다. 빨간/노란 공의 상대 위치가 위에서 본 그대로 보인다.

```
# ===== 수구 추적 -> 원본/탑뷰 궤적 프레임 생성 =====
cap = cv2.VideoCapture(VIDEO_PATH); cap.set(cv2.CAP_PROP_POS_FRAMES, START)
path = []; path_top = []; orig_frames = []; top_frames = []
prev = None; still = 0; count = 0
while count < MAX_FRAMES:
    ok, f = cap.read()
    if not ok: break
    balls = find_white_balls(f)
    if balls:
        prev = balls[0] if prev is None else min(
            balls, key=lambda p: (p[0]-prev[0])**2 + (p[1]-prev[1])**2)
        still = still + 1 if path and (prev[0]-path[-1][0])**2 + (prev[1]-path[-1][1])**2 < 9 else 0
        path.append(prev); path_top.append(to_top(prev))

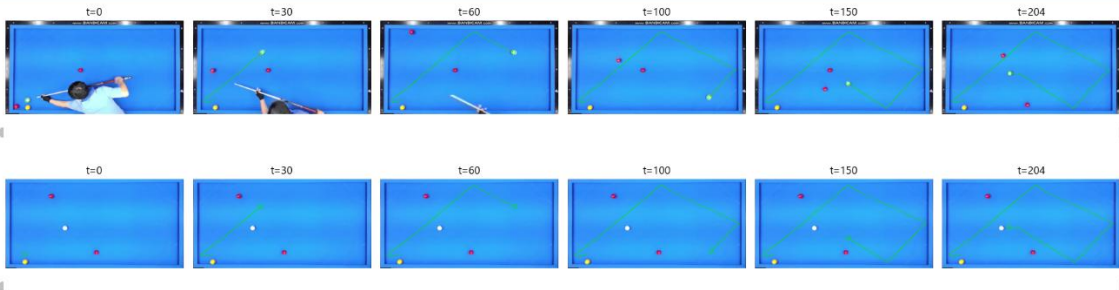
    vo = f.copy()
    if len(path) >= 2:
        cv2.polylines(vo, [np.array(path, np.int32)], False, (0, 230, 0), 2)
    if path: cv2.circle(vo, path[-1], 10, (0, 230, 0), 2)
    orig_frames.append(vo)

    vt = table_bg.copy()
    if len(path_top) >= 2:
        cv2.polylines(vt, [np.array(path_top, np.int32)], False, (0, 230, 0), 2)
    if path_top: cv2.circle(vt, path_top[-1], 12, (0, 230, 0), 2)
    top_frames.append(vt)

    count += 1
    if still > 20: break
cap.release()
print("추적 끝, 프레임:", len(orig_frames))
```

```
# ===== |출3: 추적 궤적 시간순 스냅샷 =====
SNAPSHOTS = [0, 30, 60, 100, 150, len(orig_frames) - 1]
snaps_orig = [orig_frames[i] for i in SNAPSHOTS if 0 <= i < len(orig_frames)]
snaps_top = [top_frames[i] for i in SNAPSHOTS if 0 <= i < len(orig_frames)]
labels = [f"t={i}" for i in SNAPSHOTS if 0 <= i < len(orig_frames)]

show_row(snaps_orig, labels, figsize=(18, 3)) # 원본 시점 시간순
show_row(snaps_top, labels, figsize=(18, 3)) # 탑뷰 시간순
```



수구 궤적 추적 결과 (시계열)

위: 원본 시점 / 아래: 탑뷰 시점 — t = 0, 30, 60, 100, 150, 204

시작 프레임(112)부터 매 프레임 수구를 검출·연결해 녹색 궤적을 누적했다. 현재 위치는 원으로 표시.

탑뷰(아래)에서는 원근 왜곡이 사라져, 쿠션에 맞고 꺾이는 경로가 실제 비율의 꺾인 직선으로 보인다. 원본(위)에서는 같은 경로가 원근 때문에 안쪽으로 휘어 보이는데, 이 차이가 바로 호모그래피를 넣은 이유다.

가장 고생한 실패

무엇이 안 됐나. 처음엔 흰색 이진화 → 모폴로지 → "가장 큰 흰 덩어리를 공으로" 만 했다. 그런데 스트로크 순간에 흰 큐대가 수구 바로 옆/위에 길게 누우면서, "가장 큰 흰 덩어리"가 공이 아니라 큐대가 되어 버렸다. 그 결과 추적점이 공에서 큐대로 옮겨 붙었고, 다음 프레임에 큐대가 움직이니 궤적이 공의 진짜 경로를 벗어나 큐대 방향으로 쭉 튀었다.

무엇을 관찰했나. 모든 컨투어를 박스 쳐서 보니 큐대는 면적은 공과 비슷하거나 더 컸지만 바운딩 박스가 길쭉했다(가로/세로 비가 크게 벗어남). 반면 공은 어느 프레임에서나 박스가 거의 정사각이었다. 즉 "크기"로는 공과 큐대를 못 가르치지만 "모양 비율"로는 깔끔히 갈렸다.

무엇을 바꿨더니 됐나. 컨투어 필터에 종횡비 조건 $0.55 < bw/bh < 1.8$ 을 추가했다. 그러자 길쭉한 큐대가 탈락하고 정사각에 가까운 공만 남았다 추가로, 면적 상한 2600으로 사람 옷·큰 반사 덩어리도 같이 걸러졌다. 이 한 줄을 넣은 뒤로 스트로크 구간에서도 추적점이 공에 붙어 있었다.

여기서 배운 것. 색(흰색)만으로는 공과 큐대가 구분되지 않는다 — 둘 다 흰색이니까. 구분은 기하(모양) 에서 나왔다. "무슨 색인가"와 "무슨 모양인가"를 다른 층의 단서로 나눠 쓴 게 이 실패의 핵심 교훈이다.

기법 선택의 이유

(1) 흰색 검출을 RGB가 아니라 HSV로 — $[0,0,190] \sim [180,45,255]$

HSV. 흰색을 RGB로 잡으려면 $R \approx G \approx B$ 가 모두 높다는 세 채널이 얹힌 조건이 필요해 조명에 민감하다. HSV에서는 흰색 = "채도(S) 낮고 명도(V) 높다"로 두 축만 보면 돼서 임계가 직관적이고 조명 변화에 더 견고하다.

(2) 모폴로지 타원 커널 5×5

타원 커널.을 한이유는 둥근 공 모양에 맞아 모서리를 덜 깎는다. 사각 커널은 둥근 경계에 계단 자국을 남긴다. 왜 5이고 3·15가 아니면 큐대 두께와 점 노이즈가 대략 $2 \sim 4\text{px}$ 이라 5×5 열기로 깔끔히 사라진다. 3×3 은 큐대 잔상이 일부 살아남았고, 15×15 는 공까지 같이 갉아먹거나 큐대와 붙은 영역을 합쳐 공이 뭉개졌다. 노이즈는 지우고 공은 살리는 가장 작은 크기가 5였다.

(3) 컨투어 면적·종횡비 필터 — $150 < \text{면적} < 2600, 0.55 < \text{가로/세로} < 1.8$

면적 하한 150 보다 작은 건 잔여 노이즈 점. 면적 상한 2600 보다 크면 사람 옷·큰 반사.이 해상도에서 수구는 수백~약 2천 px^2 범위였다.

(4) 호모그래피 — `getPerspectiveTransform` (4점) on 빈 프레임

빈 프레임에서 코너. 카메라가 고정이라 코너는 한 번만 찾으면 된다. 공·사람이 없는 360번 프레임을 써야 파란 영역이 가림 없이 한 덩어리로 잡혀 코너가 안정적이다.

AI 협업 부록

사례 1 — 흰 공 검출 방법

"당구 영상에서 흰 공만 검출하고 싶다." 물어보니 AI는 처음에 그레이스케일 + 오츠 이진화를 제안했다. 그대로 돌려 보니 파란 테이블·밝은 영역까지 다 흰색으로 잡혀 전혀 분리되지 않았다. 그래서 색을 직접 다루는 HSV `inRange`(낮은 S·높은 V) 로 바꿨다.

사례 2 — 탐뷰 만들기

"비스듬한 테이블을 위에서 본 것처럼 펴고 싶다." 물어보니 AI는 `findHomography` + 특징점 매칭을 제안했다. 텍스처 없는 파란 테이블에선 특징점이 안 잡혀 불안정했다. 개념(호모그래피로 펴기)은 그대로 쓰되, 구현은 코너 4점 `getPerspectiveTransform` 으로 바꿨다. `order_corners`로 코너 순서를 맞춰야 한다는 것도 직접 디버깅하며 추가했다.

사례 3 — 모폴로지 커널 크기

"노이즈를 없애려면 커널을 얼마로?" 물어보니 AI는 큰 커널(예: 15)을 제안했는데 그러면 공까지 뭉개졌다. 직접 3·5·15를 비교해 보고, 큐대·노이즈는 지우고 공은 남기는 5×5로 정했다

한계와 솔직한 미해결

가림 구간 보간 없음.

스트로크 중 큐대/손이 공을 완전히 가리면 그 프레임은 검출이 비고, 공이 다시 보일 때까지 궤적이 건너뛴다. 그래서 가림이 길면 궤적에 미세한 점프가 남는다(칼만 필터 등으로 보간하면 개선될 텐데 아직 안 넣었다).

임계값의 일반화 한계.

HSV 흰색 범위·면적·중형비 모두 이 영상의 조명과 촬영 거리에 맞춰 손으로 튜닝했다. 다른 영상엔 재튜닝이 필요하다.

코너 4점 의존.

탭뷰가 코너 4점에만 의존해서, 코너 검출이 몇 px만 틀어져도 탭뷰가 살짝 기운다. 더 견고하게 하려면 테이블 경계 직선을 여러 개 맞춰 교점을 코너로 쓰는 방법을 생각해 볼 수 있다(아직 미 적용).